

Week 2 - Python Intro

Disclaimer: this notebook is AI-generated

Notebook Basics: Markdown vs Code

- **Markdown cells:** for text, headings, math, lists.
- **Code cells:** for Python that you can run.

Note: python comment with `#` is not the same as markdown cell

Main Keyboard Shortcuts (VS Code Jupyter)

VS Code (Jupyter):

- **Add new cell:** A (above), B (below) in command mode; or `Ctrl+Shift+=`
- **Change cell type:** M (Markdown), Y (Code)
- **Delete cell:** DD (press D twice)
- **Split cell:** `Ctrl+Shift+-`
- **Run cell:** `Shift+Enter`
- **Run & advance:** `Alt+Enter`
- **Toggle command/edit mode:** `Esc / Enter`

(Windows/Linux: use `Ctrl` instead of `Cmd`.)

Topic 1 - Object Types

We'll create: **bool, int, float, str, list, tuple, dict.**

Check types with `type()`. Cast types with `int()`, `float()`, `str()` when sensible.

```
# Booleans
is_student = True
has_halbtax = False
print(is_student, type(is_student))

# Integers & floats
population_ch = 8_900_000      # ~8.9M
inflation = 1.6                # %
print(type(population_ch), type(inflation))
```

```

# Strings
city = "Zürich"
currency = "CHF"
greeting = f"Welcome to {city}!"
print(greeting, type(greeting))

# Lists (mutable)
cantons = ["ZH", "BE", "LU", "UR"]
cantons.append("VD")
print(cantons, type(cantons))

# Tuples (immutable)
coords_uni_zh = (47.376, 8.548)
print(coords_uni_zh, type(coords_uni_zh))

# Dictionaries (key → value)
gdp = {"CH": 800, "DE": 4500, "LT": 70} # CHF billions (illustrative)
print(gdp, type(gdp))

# Type casting
year_str = "2025"
year_int = int(year_str)
pi_str = "3.14"
pi_float = float(pi_str)
print(year_int, type(year_int), "|", pi_float, type(pi_float))

```

Mini-exercise

Create your own: a `str` with your **major**, an `int` with **ECTS**, and a `dict` mapping **course code** → **ECTS**. Print their `type()`.

Topic 2 - Indexing & Slicing

Access elements inside: **strings**, **lists/tuples**, and **dicts**.

- Zero-based indexing: first item is index 0.
- Slicing: `seq[start:stop:step]` (stop is exclusive).

```

# Strings
sbb_line = "IC5: Zürich HB → Lausanne"
print(sbb_line[0], sbb_line[-1]) # first and last char
print(sbb_line[4:10])           # slice
print(sbb_line[::-1])           # reverse

# Lists / tuples
fares = [3.4, 4.2, 8.8, 12.5]    # CHF
print(fares[0], fares[1:3], fares[-1])
print(coords_uni_zh[0])         # tuple indexing

# Dictionaries
print(gdp["CH"])                # by key
gdp["CH"] = gdp["CH"] * 1.01    # update value
print(gdp.get("FR", "No data")) # safe get with default

```

Mini-exercise

Given `name = "University of Zurich"`, slice to get `"Zurich"`. From `cantons`, get the last two entries via slicing.

Topic 3 - Basic Operations & Built-in Functions

- **Math** on numbers: + - / // % *

- **Strings**: concatenation +, repeat *, .upper(), .lower(), .replace()

- **Lists**: len, sorted, .append(), .extend()

```
# Numbers
monthly_ticket = 70    # CHF (example)
months = 6
cost_semester = monthly_ticket * months
print("Semester cost:", cost_semester)

# Strings
school = "UZH"
msg = school + " Economics"
print(msg, "| upper:", msg.upper())

# Lists: length & sorting
prices = [12.5, 3.4, 8.8, 4.2]
print("n =", len(prices), "| sorted:", sorted(prices))

# Mutating list
prices.append(6.9)
print("updated:", prices)
```

Mini-exercise

Create a list of three **favorite Swiss cities**, then print how many and the sorted list alphabetically.

Topic 4 - Booleans & Conditionals

Use comparisons to produce booleans, then branch with `if / elif / else`.

Common comparisons: `==, !=, >, <, >=, <=`; combine with `and, or, not`.

```
# Simple policy example: student discount if age < 25 OR has Halbtax
age = 23
has_halbtax = True

eligible = (age < 25) or has_halbtax
print("Discount eligible?", eligible)

# If/elif/else
income = 2200    # CHF/month (toy example)
if income < 2000:
    bracket = "low"
elif income < 5000:
    bracket = "middle"
else:
    bracket = "high"
print("Income bracket:", bracket)
```

Topic 5 - File Input/Output

```
# Use the existing data folder at the root of week_2
from pathlib import Path

# Paths are relative to this notebook's intro folder
DATA = Path("../data")
in_path = DATA / "cities.txt"
out_path = DATA / "cities_clean.txt"
```

```
# 1) Read the .txt file
raw_text = in_path.read_text(encoding="utf-8")
lines = raw_text.splitlines()

# 2) Clean & transform (strip empties, normalize spacing, Title Case)
cleaned = []
for ln in lines:
    ln = ln.strip()
    if not ln:
        continue
    cleaned.append(ln.title())

# 3) Save to a new .txt
out_path.write_text("\n".join(cleaned), encoding="utf-8")

# 4) Quick preview
print(f"Input file: {in_path} ({len(lines)} lines)")
print(f"Output file: {out_path} ({len(cleaned)} lines)\n")
print("Preview:")
for ln in cleaned[:5]:
    print(" •", ln)
```

Topic 6 - Importing Libraries & Local Scripts

Use `import` to access reusable code. An alias such as `np` gives a library a shorter name; use `name.function()` to call its functions.

NumPy must be installed in your notebook environment (if needed, run `%pip install numpy` in a separate cell). `math` comes with Python.

```
# Import two libraries and call their functions
import numpy as np
import math

prices = [3.4, 4.2, 8.8, 12.5]
print("Average price:", np.mean(prices))
print("Square root of 81:", math.sqrt(81))
```

A local `.py` file can also be imported as a **module**. The companion `my_module.py` is in this notebook's folder; run with `intro` as the working directory.

`from ... import ...` imports selected variables; `import ...` imports the whole module, whose variables and functions you access with a dot. Omit `.py` in both forms.

Python runs a module's top-level code on its first import in a kernel session.

```
# Import selected variables from the local script
from my_module import city, ticket_price

print("City:", city)
print("Ticket price:", ticket_price, "CHF")

# Import the whole local script as a module
import my_module

print("Currency:", my_module.currency)
print("Cost of 3 tickets:", my_module.total_cost(3), my_module.currency)
```